Python 技巧和竅門

本筆記本涵蓋以下 Python 編程技巧：

1. 如何讓 Python 代碼更易讀
2. 使用 `Context Manager` 管理資源
3. 使用 `try-except-else-finally` 進行安全的異常處理
4. 使用 `enumerate` 進行計數
5. 使用 `zip` 進行平行迭代
6. 使用解包(unpacking)將物件賦值給變數
7. 使用 `Setattr` / `Getattr` 動態設定/獲取物件屬性
8. 使用 `getpass` 從用戶端獲取密碼
9. 使用 `python -m` 將模組作為腳本執行
10. 使用 `help` / `dir` 查看文檔字串和物件屬性
11. 使用 `Iterable` 進行迭代

Python 環境設定

In [1]:
```
import sys
print(f'Python sys.version:\n{sys.version}\n')
print(f'Python sys.executable:\n{sys.executable}\n')
print(f'Python sys.path:\n{sys.path}\n')
```

```
Python sys.version:
3.13.2 | packaged by Anaconda, Inc. | (main, Feb  6 2025, 12:55:35)
[Clang 14.0.6 ]

Python sys.executable:
/Users/michaelkao/miniconda3/envs/pex/bin/python

Python sys.path:
['/Users/michaelkao/miniconda3/envs/pex/lib/python313.zip', '/Users/
michaelkao/miniconda3/envs/pex/lib/python3.13', '/Users/michaelkao/m
iniconda3/envs/pex/lib/python3.13/lib-dynload', '', '/Users/michaelk
ao/miniconda3/envs/pex/lib/python3.13/site-packages']
```

In [2]:
```
import os
```

```
print(f'Current Working Dir (CWD) path:\n{os.getcwd()}\n')
```

```
Current Working Dir (CWD) path:
/Users/michaelkao/My_Projects/Python
```

1. 如何讓 Python 代碼更易讀

1.1 使用描述性的變數和函數名稱

選擇能夠清楚描述其內容或功能的變數和函數名稱。 例如，與其使用 a = 10，不如使用 user_age = 10。 清晰的名稱讓您的代碼不言自明，減少註解的需求。

In [3]:
```
# 不好的寫法
a = 5
b = 2

# 好的寫法
total = 5
sub_total = 2
print("總數:   ", total)
print("小計:", sub_total)
```

```
總數:    5
小計: 2
```

In [4]:
```
# 不好的寫法
def a():
    return 5

def b():
    return 2

# 好的寫法
def calculate_total():
    return 5

def calculate_sub_total():
    return 2

print("調用 calculate_total()     =", calculate_total())
print("調用 calculate_sub_total() =", calculate_sub_total())
```

```
調用 calculate_total()     = 5
調用 calculate_sub_total() = 2
```

1.2 Use ternary conditionals (三元條件運算)

Python 使用三元運算子(ternary conditional operator) 好處是程式碼可以看起來比較簡短，在某些情況 python 使用三元運算子簡化後 code 會變得清爽簡短許多。

C 的寫法:

```
x = condition? 1 : 0
```

Python 的寫法:

```
x = 1 if condition else 0
```

Python 二元運算的寫法較多行但邏輯相同，如下：

```
if condition:
    x = 1
else:
    x = 0
```

In [5]:
```
condition = False

x = 1 if condition else 0

# 使用 f-strings 進行格式化
# 使用 f-strings 可以讓字串插值更清晰易讀。
# 它們比字串連接更快且更優雅。

print(f"條件 = {condition}")
print(f"x = {x}")

print(f'條件為真') if condition else print('條件為假')
```

```
條件 = False
x = 0
條件為假
```

1.3 Python 的 match 語句說明

match 是 Python 3.10 引入的結構化模式匹配語法，它提供了一種更強大、更直觀的方式來進行條件分支處理，特別適合處理複雜的數據結構。

基本語法:

```
match subject:
    case pattern1:
        # 處理 pattern1 的情況
    case pattern2:
        # 處理 pattern2 的情況
    case _:
        # 預設情況 (類似於 switch 中的 default)
```

優點:

- 比傳統的 if-elif-else 鏈更簡潔
- 支持結構化數據的深度匹配
- 更易讀，特別是在處理複雜的數據結構時
- 可以與類型提示結合使用，提高代碼的可讀性和可維護性

注意事項:

- match 是 Python 3.10 的新特性，舊版本不支援
- 模式匹配是按順序進行的，第一個匹配的模式會被執行
- "_" 是萬用字元，可以匹配任何值
- 模式可以嵌套，允許複雜的數據結構匹配
- match 語句特別適合處理 JSON 數據、解析命令列參數、處理 AST（抽象語法樹）等場景。

程式碼說明：

1. 函數功能：

- describe_type() 函數使用 Python 3.10+ 的 match-case 語法來判斷輸入值的類型
- 根據不同類型返回對應的中文描述

2. 支援的類型：

- 布林值（bool）：顯示「真」或「假」
- 整數（int）：區分正整數和負整數
- 字串（str）：顯示長度和內容
- 列表（list）：顯示元素數量
- 集合（set）：顯示不重複元素的數量
- 元組（tuple）：顯示元素數量
- 字典（dict）：顯示鍵值對數量
- 空值（None）
- 其他類型：顯示類型名稱

3. 測試案例：

- 包含各種常見的 Python 數據類型
- 每個測試案例都展示了不同的類型判斷情況
- 輸出結果會顯示原始值及其對應的類型描述

4. 輸出格式：

- 每行顯示一個測試值及其類型描述
- 使用字串格式化對齊輸出，提高可讀性

In [6]:
```
def describe_type(value):
    """
    根據輸入值的類型返回相應的描述字串

    Args:
        value: 要檢查類型的值，可以是任何Python對象

    Returns:
        str: 描述值類型和特性的字串
    """
```

```
    match value:
        case bool():
            return f"布林值: {'真' if value else '假'}"
        case int() if value >= 0:
            return f"非負整數: {value}"
        case int():  # 負整數
            return f"負整數: {value}"
        case str():
            length = len(value)
            if length == 0:
                return "空字串"
            return f"長度為 {length} 的字串: '{value}'"
        case list():
            length = len(value)
            return f"包含 {length} 個元素的列表"
        case set():
            length = len(value)
            return f"包含 {length} 個不重複元素的集合"
        case tuple():
            length = len(value)
            return f"包含 {length} 個元素的元組"
        case dict():
            length = len(value)
            return f"包含 {length} 個鍵值對的字典"
        case None:
            return "空值 (None)"
        case _:
            return f"其他類型: {type(value).__name__}"

# 測試案例
test_values = [
    42,                     # 正整數
    -7,                     # 負整數
    True,                   # 布林值 True
    False,                  # 布林值 False
    "",                     # 空字串
    "hello",                # 一般字串
    [1, 2, 3],              # 列表
    {1, 2, 3, 3},           # 集合（自動去重）
    (10, 20),               # 元組
    {"name": "Alice", "age": 25},  # 字典
    None,                   # 空值
    3.14                    # 浮點數（會進入預設情況）
]

print("類型判斷範例：\n" + "="*30)
for val in test_values:
    print(f"{val!r:30} → {describe_type(val)}")
```

類型判斷範例：

```
==============================
42                              → 非負整數: 42
-7                              → 負整數: -7
True                            → 布林值: 真
False                           → 布林值: 假
''                              → 空字串
'hello'                         → 長度為 5 的字串: 'hello'
[1, 2, 3]                       → 包含 3 個元素的列表
{1, 2, 3}                       → 包含 3 個不重複元素的集合
(10, 20)                        → 包含 2 個元素的元組
{'name': 'Alice', 'age': 25}    → 包含 2 個鍵值對的字典
None                            → 空值 (None)
3.14                            → 其他類型: float
```

1.4 使用 _（底線）作為忽略的變數或佔位符

當您有一個數值變量，其值不容易閱讀時，可以使用 _ 來表示千位分隔，以提高可讀性。

```
# 不好的寫法
num = 1000000

# 好的寫法
num = 1_000_000
```

或者當您不需要使用循環變量時，可以使用 _ 來表示這個變量是被故意忽略的。這是一個簡單但強大的約定。

```
for _ in range(3):
    print("Hello, World!")
```

In [7]:
```
num1 = 10000000000
num2 = 100000000

total = num1 + num2
print(f"總和 = {total}")
```

```
總和 = 10100000000
```

In [8]:
```
# 使用 "_" 提升可讀性（無操作）
num1 = 10_000_000_000
num2 = 100_000_000

total = num1 + num2
print(f"總和 = {total}")

# 使用 "_" 表示未使用的循環計數器
for _ in range(3):
    print("Hello World!")
```

```
總和 = 10100000000
Hello World!
Hello World!
Hello World!
```

In [9]:
```
# 使用 ',' 格式化提升報表可讀性
print(f'總和 = {total:,}')
```

```
總和 = 10,100,000,000
```

1.5 使用列表推導式創建列表

列表推導式是一種簡潔的創建列表的方式。與傳統的循環相比，它們使您的代碼更簡短且更易於理解。

```
squares = [x * x for x in range(10)]
```

In [10]:
```
# 建立一個包含 0 到 9 的平方數列表
squares = [x * x for x in range(10)]
print(f"平方數列表: {squares}")
```

```
平方數列表: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

In [11]:
```
# 從列表中找出偶數
nums = [1, 2, 3, 4, 5, 6]
evens = [num for num in nums if num % 2 == 0]
print(f"偶數: {evens}")
```

```
偶數: [2, 4, 6]
```

1.6 不使用暫存變數交換兩個變數

C 的寫法:

```
int a = 1;
int b = 2;

temp = a;
a = b;
b = temp;
```

Python 的寫法:

```
a = 1
b = 2

a, b = b, a
```

In [12]:
```
# 交換變數
a = 1
b = 2
a, b = b, a
print(f"a = {a}, b = {b}")
```

```
a = 2, b = 1
```

1.7 字串處理

- 反轉字串

- 檢查兩個字串是否為變位詞
- 計算字串中字符的出現頻率
- 移除字串中的重複字符

In [13]:
```
# 反轉字串
god = "god"
print(f"反轉 god: {god[::-1]}")

stressed = "stressed"
print(f"反轉 stressed: {stressed[::-1]}")
```

```
反轉 god: dog
反轉 stressed: desserts
```

In [14]:
```
# 檢查兩個字串是否為變位詞
str_1 = "listen"
str_2 = "silent"

print(sorted(str_1) == sorted(str_2))
```

```
True
```

In [15]:
```
# 計算字串中字符的出現頻率
from collections import Counter
print (Counter([1,2,2,2,3,3,3,3,3]))
```

```
Counter({3: 5, 2: 3, 1: 1})
```

In [16]:
```
# 移除字串中的重複字符
def remove_duplicates(input_str):
    # 每個字符將成為字典的鍵，這樣可以消除重複
    # 值並不重要，所以我們使用 None
    return ''.join(dict.fromkeys(input_str))

# 使用範例
text = "hello world"
result = remove_duplicates(text)  # "helo wrd"
print(f"原始字串: {text}")
print(f"移除字串中的重複字符結果: {result}")
```

```
原始字串: hello world
移除字串中的重複字符結果: helo wrd
```

1.8 列表處理

- 反轉列表
- 移除列表中的重複項
- 對列表進行排序
- 展平嵌套列表

In [17]:
```
# 反轉串列
my_list = [1, 2, 3, 4, 5]
print(f"原始串列: {my_list}")
reversed_list = my_list[::-1]
```

```
print(f"反轉後的串列: {reversed_list}")

# 移除串列中的重複元素
my_list = [1, 2, 2, 3, 3, 3, 4, 5]
unique_list = list(set(my_list))
print(f"去重複元素後的串列: {unique_list}")

# 排序串列
my_list = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
sorted_list = sorted(my_list)
print(f"排序後的串列: {sorted_list}")
```

```
原始串列: [1, 2, 3, 4, 5]
反轉後的串列: [5, 4, 3, 2, 1]
去重複元素後的串列: [1, 2, 3, 4, 5]
排序後的串列: [1, 1, 2, 3, 3, 4, 5, 5, 5, 6, 9]
```

In [18]:
```
# 將多維串列扁平化
lists = [[1, 2, 3], [4,5], [6, 7, 8]]
print(f"原始多維串列: {lists}")
flat_list = [x for row in lists for x in row]
print(f"扁平化後的串列: {flat_list}")
```

```
原始多維串列: [[1, 2, 3], [4, 5], [6, 7, 8]]
扁平化後的串列: [1, 2, 3, 4, 5, 6, 7, 8]
```

1.9 字典處理

- 遍歷字典
- 提取子字典

In [19]:
```
student = {
        'name': 'John',
        'age': 25,
        'courses': ['Math', 'ComSci']
    }
# 使用 items() 方法遍歷
print("使用 items() 方法遍歷:")
for key, value in student.items():
    print(f"\t{key}: {value}")

# 使用 keys() 方法遍歷
print("\n使用 keys() 方法遍歷:")
for key in student.keys():
    print(f"\t{key}")

# 使用 values() 方法遍歷
print("\n使用 values() 方法遍歷:")
for value in student.values():
    print(f"\t{value}")
```

```
使用 items() 方法遍歷:
        name: John
        age: 25
        courses: ['Math', 'ComSci']

使用 keys() 方法遍歷:
        name
        age
        courses

使用 values() 方法遍歷:
        John
        25
        ['Math', 'ComSci']
```

In [20]:
```python
# 使用字典推導式建立子字典
original_dict = {
        'name': 'John',
        'age': 30,
        'city': 'New York',
        'occupation': 'Engineer',
        'email': 'john@example.com'
    }
print(f"原始字典: {original_dict}")

# 子字典中的鍵
keys = ['name', 'age', 'city']
sub_dict = {key: original_dict[key] for key in keys if key in origi
print(f"子字典: {sub_dict}")
```

```
原始字典: {'name': 'John', 'age': 30, 'city': 'New York', 'occupatio
n': 'Engineer', 'email': 'john@example.com'}
子字典: {'name': 'John', 'age': 30, 'city': 'New York'}
```

2. 使用 `Context Manager` 管理資源

當我們讀取或寫入檔案時，有時會忘記在最後關閉它。這無疑會浪費系統資源，例如檔案物件和緩衝區，一旦分配後就無法重複使用。

Python 通過 "with" 語句提供了一個上下文管理器，當您退出 "with" 區塊的執行時，它會自動為您管理資源。對於檔案操作，請始終使用 with open()。它能確保檔案自動關閉，防止資源洩漏或錯誤。

以下是名為 "test.txt" 的範例檔案內容：

```
1) This is a test file
2) With multiple lines of data...
3) Third line
4) Fourth line
5) Fifth line
6) Sixth line
7) Seventh line
8) Eighth line
```

```
9) Ninth line
10) Tenth line
```

In [21]:
```python
# 開啟檔案的各種模式：
# f = open("test.txt", "r")  # 唯讀模式
# f = open("test.txt", "w")  # 寫入模式（會覆蓋原有內容）
# f = open("test.txt", "a")  # 附加模式（在檔案末尾添加內容）
# f = open("test.txt", "r+") # 讀寫模式

f = open('sample/test.txt', 'r')
print(f'檔案物件:\t {f.name}')
print(f'---> 開啟模式:\t {f.mode}\n')

file_contents = f.read()
f.close()
print(f'---> 檔案已關閉?\t {f.closed}\n')


print(f'將整個檔案讀取到一個字串變數中:\n{file_contents}\n')

words = file_contents.split(' ')
print(f'檔案 "test.txt" 中的單字:\n{words}\n')

word_count = len(words)
print(f'檔案 "test.txt" 中共有 {word_count} 個單字。\n')
```

```
檔案物件:          sample/test.txt
---> 開啟模式:     r

---> 檔案已關閉?   True

將整個檔案讀取到一個字串變數中:
1) Here is a joke
2) With 6 lines 45 words, it says:
3) Teacher: How old is your father, Henry?
4) Henry: He is 6 years old.
5) Teacher: What? How is this possible?
6) Henry: He became father only when I was born.


檔案 "test.txt" 中的單字:
['1)', 'Here', 'is', 'a', 'joke\n2)', 'With', '6', 'lines', '45', 'w
ords,', 'it', 'says:\n3)', 'Teacher:', 'How', 'old', 'is', 'your', '
father,', 'Henry?\n4)', 'Henry:', 'He', 'is', '6', 'years', 'old.\n
5)', 'Teacher:', 'What?', 'How', 'is', 'this', 'possible?\n6)', 'Hen
ry:', 'He', 'became', 'father', 'only', 'when', 'I', 'was', 'born.\
n']

檔案 "test.txt" 中共有 40 個單字。
```

2.1 使用 `Context Manager` 讀取檔案

常見的讀取方法有：

1. f.read()
2. f.readlines()
3. f.readline()
4. f.seek(位置)
5. f.tell()

以及屬性：

1. f.mode
2. f.closed

In [22]:
```
# 使用 'with' 上下文管理器讀取檔案
# 不僅適用於檔案，還適用於其他資源，如線程

with open('sample/test.txt', 'r') as f:

    print(f'檔案物件:\t {f.name}')
    print(f'---> 開啟模式:\t {f.mode}\n')

    # 將整個檔案內容讀取到一個字串變數中
    file_contents = f.read()
    print(f'已將整個檔案讀取到一個字串變數中:\n{file_contents}\n')


# 上下文管理器在離開 with 區塊後會自動關閉檔案
print(f'---> 上下文管理器已結束。檔案是否已關閉?\t {f.closed}\n')

try:
    # 嘗試在檔案關閉後讀取
    no_contents = f.read()

except ValueError:
    print('f.read(): 在已關閉的檔案上進行 I/O 操作。\n')
except:
    print('發生其他錯誤。\n')


words = file_contents.split(' ')
print(f'查看 "test.txt" 檔案中的單字:\n{words}\n')

word_count = len(words)
print(f'檔案 "test.txt" 中共有 {word_count} 個單字。\n')
```

```
檔案物件:          sample/test.txt
---> 開啟模式:     r

已將整個檔案讀取到一個字串變數中:
1) Here is a joke
2) With 6 lines 45 words, it says:
3) Teacher: How old is your father, Henry?
4) Henry: He is 6 years old.
5) Teacher: What? How is this possible?
6) Henry: He became father only when I was born.


---> 上下文管理器已結束。檔案是否已關閉?       True

f.read(): 在已關閉的檔案上進行 I/O 操作。

查看 "test.txt" 檔案中的單字:
['1)', 'Here', 'is', 'a', 'joke\n2)', 'With', '6', 'lines', '45', 'w
ords,', 'it', 'says:\n3)', 'Teacher:', 'How', 'old', 'is', 'your', '
father,', 'Henry?\n4)', 'Henry:', 'He', 'is', '6', 'years', 'old.\n
5)', 'Teacher:', 'What?', 'How', 'is', 'this', 'possible?\n6)', 'Hen
ry:', 'He', 'became', 'father', 'only', 'when', 'I', 'was', 'born.\
n']

檔案 "test.txt" 中共有 40 個單字。
```

2.2 更多關於檔案讀取方法的範例

In [23]:

```python
# 更多讀取檔案的方法
with open('sample/test.txt', 'r') as f:

    # 將檔案內容讀取到一個行列表中
    f_contents = f.readlines()
    print(f'一次讀取所有行到列表中:\n{f_contents}\n')

    # 回到檔案開頭
    f.seek(0)

    # 一次讀取一行
    f_contents = f.readline()
    print(f'讀取第一行到列表中:\n{f_contents}')

    f_contents = f.readline()
    print(f'讀取第二行到列表中:\n{f_contents}')

    # 忽略換行符
    f_contents = f.readline()
    print(f'讀取第三行到列表中，忽略換行:\n{f_contents}', end='')
    f_contents = f.readline()
    print(f'讀取第四行到列表中，忽略換行:\n{f_contents}', end='')

    # 使用 for 迴圈逐行讀取，直到檔案結尾
    for line in f:
        print(f'使用 for 迴圈逐行讀取，直到檔案結尾...\n->{line}<-\n', en
```

```
    # 回到檔案開頭 (BOF)
    f.seek(0)

    # 使用 while 迴圈每次讀取指定字節數，直到檔案結尾 (EOF)
    size_to_read = 100
    f_contents = f.read(size_to_read)
    while len(f_contents) > 0:
        print(f'\n使用 while 迴圈，每次讀取 {size_to_read} 字節，直到檔案
        f_contents = f.read(size_to_read)

    # 再次回到檔案開頭
    f.seek(0)

    # 以較小的區塊讀取，每次 10 個字元
    size_to_read = 10
    f_contents = f.read(size_to_read)
    print(f'讀取 {size_to_read} 字節，忽略換行:\n->{f_contents}<-\n', 
    print(f'當前檔案物件位置 = {f.tell()}\n')

    f.seek(0)
    f_contents = f.read(size_to_read)
    print(f'回到檔案開頭，讀取 {size_to_read} 字節，忽略換行:\n->{f_conte

    print(f'當前檔案物件位置 = {f.tell()}\n')
```

```
一次讀取所有行到列表中:
['1) Here is a joke\n', '2) With 6 lines 45 words, it says:\n', '3)
Teacher: How old is your father, Henry?\n', '4) Henry: He is 6 years
old.\n', '5) Teacher: What? How is this possible?\n', '6) Henry: He
became father only when I was born.\n']

讀取第一行到列表中:
1) Here is a joke

讀取第二行到列表中:
2) With 6 lines 45 words, it says:

讀取第三行到列表中，忽略換行:
3) Teacher: How old is your father, Henry?
讀取第四行到列表中，忽略換行:
4) Henry: He is 6 years old.
使用 for 迴圈逐行讀取，直到檔案結尾...
->5) Teacher: What? How is this possible?
<-
使用 for 迴圈逐行讀取，直到檔案結尾...
->6) Henry: He became father only when I was born.
<-

使用 while 迴圈，每次讀取 100 字節，直到檔案結尾...
->1) Here is a joke
2) With 6 lines 45 words, it says:
3) Teacher: How old is your father, Henry?
4) H<-

使用 while 迴圈，每次讀取 100 字節，直到檔案結尾...
->enry: He is 6 years old.
5) Teacher: What? How is this possible?
6) Henry: He became father only whe<-

使用 while 迴圈，每次讀取 100 字節，直到檔案結尾...
->n I was born.
<-
讀取 10 字節，忽略換行:
->1) Here is<-
當前檔案物件位置 = 10

回到檔案開頭，讀取 10 字節，忽略換行:
->1) Here is<-
當前檔案物件位置 = 10
```

2.3 使用 `Context Manager` 寫入檔案

In [24]:

```python
# 使用上下文管理器寫入檔案
with open("sample/test2.txt", "w") as f:
    f.write("Testing123")
    f.seek(0)
    f.write("Test987\n")

with open("sample/test2.txt", "r") as f:
```

```
    # 逐行讀取，直到檔案結尾
    for line in f:
        print(f'逐行讀取，直到檔案結尾...\n->{line}<-\n', end='')
```

```
逐行讀取，直到檔案結尾...
->Test987
<-
逐行讀取，直到檔案結尾...
->23<-
```

2.4 使用 `Context Manager` 複製檔案

In [25]:
```
# 複製檔案到另一個檔案
with open("sample/test.txt", "r") as rf:
    with open("test_copy.txt", "w") as wf:
        for line in rf:
            wf.write(line)

file_read = "test_copy.txt"
with open(file_read, "r") as rf:
    print(f'開啟 {file_read} 檔案:\n')
    for line in rf:
        print(f'逐行讀取，直到檔案結尾...\n->{line}<-\n', end='')

# 複製圖片：以二進位模式讀取/寫入
with open("sample/Olisan.JPG", "rb") as rf:
    with open("sample/Olisan_copy.JPG", "wb") as wf:
        for line in rf:
            wf.write(line)

# 以區塊方式複製圖片
with open("sample/Olisan.JPG", "rb") as rf:
    with open("sample/Olisan_copy2.JPG", "wb") as wf:
        chunk_size = 4096
        rf_chunk = rf.read(chunk_size)

        while len(rf_chunk) > 0:
            wf.write(rf_chunk)
            rf_chunk = rf.read(chunk_size)

# 顯示所有圖片
from PIL import Image
img = Image.open('sample/Olisan.JPG')
img.show()
img_copy = Image.open('sample/Olisan_copy.JPG')
img_copy.show()
img_copy2 = Image.open('sample/Olisan_copy2.JPG')
img_copy2.show()
```

```
開啟 test_copy.txt 檔案:

逐行讀取，直到檔案結尾...
->1) Here is a joke
<-
逐行讀取，直到檔案結尾...
->2) With 6 lines 45 words, it says:
<-
逐行讀取，直到檔案結尾...
->3) Teacher: How old is your father, Henry?
<-
逐行讀取，直到檔案結尾...
->4) Henry: He is 6 years old.
<-
逐行讀取，直到檔案結尾...
->5) Teacher: What? How is this possible?
<-
逐行讀取，直到檔案結尾...
->6) Henry: He became father only when I was born.
<-
```

3. 使用 `try-except-else-finally` 進行錯誤處理

使用 `try/except` 優雅地處理運行時錯誤。 它可以防止您的代碼意外崩潰。

In [26]:
```python
try:
    # 可能會引發異常的代碼
    result = 10 / 0
except ZeroDivisionError:
    # 處理特定的異常
    print("錯誤：不能除以零")
else:
    # 如果沒有發生異常，則運行此代碼
    print("else: 除法運算成功")
finally:
    # 無論是否發生異常，此代碼都會執行
    print("finally: 這行代碼總會執行")
```

```
錯誤：不能除以零
finally: 這行代碼總會執行
```

4. 使用 `enumerate` 進行計數

在遍歷列表時，使用 enumerate() 而不是手動計數器。它會自動提供索引和值，提高代碼清晰度。

In [27]:
```python
# 使用計數器的舊方法
names = ['Corey', 'Michael', 'Travis', 'Dave']

index = 0
```

```
for name in names:
    print(index, name)
    index += 1
```

```
0 Corey
1 Michael
2 Travis
3 Dave
```

In [28]:
```
# 使用 enumerate 的更簡潔代碼
names = ['Corey', 'Michael', 'Travis', 'Dave']

for index, name in enumerate(names):
    print(index, name)
```

```
0 Corey
1 Michael
2 Travis
3 Dave
```

In [29]:
```
# 另一個使用 enumerate 的更簡潔代碼，從 1 開始計數
names = ['Corey', 'Michael', 'Travis', 'Dave']

# 將起始索引設為 1
for index, name in enumerate(names, start=1):
    print(index, name)
```

```
1 Corey
2 Michael
3 Travis
4 Dave
```

5. 使用 `zip` 處理多個物件

使用 zip() 整齊地組合多個列表。非常適合並行遍歷相關數據。

In [30]:
```
names = ['Corey', 'Michael', 'Travis', 'Dave']
heroes = ['Spiderman', 'Superman', 'Deadpool', 'Batman']
for index, name in enumerate(names):
    hero = heroes[index]
    print(f'{name} 實際上是 {hero}')
```

```
Corey 實際上是 Spiderman
Michael 實際上是 Superman
Travis 實際上是 Deadpool
Dave 實際上是 Batman
```

In [31]:
```
names = ['Corey', 'Michael', 'Travis', 'Dave']
heroes = ['Spiderman', 'Superman', 'Deadpool', 'Batman']

# 使用 zip 的更簡潔代碼
for name, hero in zip(names, heroes):
    print(f'{name} 實際上是 {hero}')
```

```
Corey 實際上是 Spiderman
Michael 實際上是 Superman
Travis 實際上是 Deadpool
Dave 實際上是 Batman
```

In [32]:
```python
names = ['Corey', 'Michael', 'Travis', 'Dave']
heroes = ['Spiderman', 'Superman', 'Deadpool', 'Batman']
universes = ['Marvel', 'DC', 'Tokyo', 'Taiwan']

# 使用 zip 合併多個列表的更簡潔代碼
for name, hero, universe in zip(names, heroes, universes):
    print(f'{name} 實際上是來自 {universe} 的 {hero}')
```

```
Corey 實際上是來自 Marvel 的 Spiderman
Michael 實際上是來自 DC 的 Superman
Travis 實際上是來自 Tokyo 的 Deadpool
Dave 實際上是來自 Taiwan 的 Batman
```

6. 使用解包(unpacking)將物件賦值給變數

它允許您在一行程式碼中將值分配給多個變數。

In [33]:
```python
# 元組的基本解包
a, b = (1, 2)
print(f"a: {a}")
print(f"b: {b}")
```

```
a: 1
b: 2
```

In [34]:
```python
# 解包列表
x, y = [10, 20]
print(f"x: {x}, y: {y}")
```

```
x: 10, y: 20
```

In [35]:
```python
# 在函數參數中進行解包
def add(a, b, c):
    return a + b + c

args = (1, 2, 3)
print(f"add(*args) = {add(*args)}")
```

```
add(*args) = 6
```

In [36]:
```python
# 解包字串（任何可迭代對象 iterable objects，字串由一系列字符組成）
a, b, c = "abc"
print(f"a={a}, b={b}, c={c}")
```

```
a=a, b=b, c=c
```

In [37]:
```python
# 解包字典，使用 ** 來解包鍵值對
def greet(name, age):
    print(f"你好，{name}。你今年 {age} 歲。")

person = {"name": "John", "age": 30}
```

```
greet(**person)

# 等同於
greet(name="John", age=30)
```

```
你好，John。你今年 30 歲。
你好，John。你今年 30 歲。
```

In [38]:
```
# 從字典中解包鍵或值
d = {'a': 1, 'b': 2, 'c': 3}

k1, k2, k3 = d.keys()
v1, v2, v3 = d.values()

print(f"k1={k1}, k2={k2}, k3={k3}")
print(f"v1={v1}, v2={v2}, v3={v3}")
```

```
k1=a, k2=b, k3=c
v1=1, v2=2, v3=3
```

In [39]:
```
# 使用 * 進行擴展解包 (Python3+)
a, b, *c, d = (1, 2, 3, 4, 5, 6)
print(f"a: {a}")
print(f"b: {b}")
print(f"c: {c}")
print(f"d: {d}")
```

```
a: 1
b: 2
c: [3, 4, 5]
d: 6
```

In [40]:
```
# 使用 *_ 來忽略多餘的解包元素 (Python3+)
a, b, *_, d = (1, 2, 3, 4, 5, 6)
print(f"a: {a}")
print(f"b: {b}")
print(f"d: {d}")
```

```
a: 1
b: 2
d: 6
```

In [41]:
```
# 巢狀解包
coords =[(1,2),(3,4),(5,6)]
x,y = zip(*coords)
print(f"x: {x}")
print(f"y: {y}")
```

```
x: (1, 3, 5)
y: (2, 4, 6)
```

In [42]:
```
# 更複雜的巢狀列表
data = ['Michael', (25, 'Engineer')]

# 解包
name, (age, profession) = data
print(f"姓名: {name}, 年齡: {age}, 職業: {profession}")
```

```
姓名: Michael, 年齡: 25, 職業: Engineer
```

7. 使用 `setattr`/`getattr` 動態設定/獲取物件屬性

動態設定和獲取物件的屬性。

In [43]:
```
class Person():
    pass

person = Person()

# 設定屬性和值
person.first = "Michael"
person.last = "Kao"

print(f'first: {person.first}, last: {person.last}')
```

```
first: Michael, last: Kao
```

In [44]:
```
class Person():
    pass

person = Person()

# 使用變數值動態設定屬性
key = "first"
value = "Michael"
person.last = "Kao"

setattr(person, key, value)

print(f'first: {person.first}, last: {person.last}')
```

```
first: Michael, last: Kao
```

In [45]:
```
class Person():
    pass

person = Person()

# 使用變數值動態設定屬性
key = "first"
value = "Michael"
person.last = "Kao"

setattr(person, key, value)

# 使用變數值獲取屬性值
first = getattr(person, key)
print(f"first: {first}")
```

```
first: Michael
```

In [46]:
```
class Person():
    pass
```

```
person = Person()

person_dict = {'first':'Michael', 'last':'Kao'}

for key, value in person_dict.items():
    setattr(person, key, value)

print(f'{person.first} {person.last}')

for key in person_dict.keys():
    print(getattr(person, key))
```

```
Michael Kao
Michael
Kao
```

8. 使用 `getpass` 向用戶請求密碼

向用戶請求密碼，並且不會在終端上顯示輸入內容。

In [47]:
```
print('登入中...')
username = input('使用者名稱: ')
password = input('密碼: ')
print(f"使用者名稱: {username} 已登入")
```

```
登入中...
使用者名稱: mkao 已登入
```

In [48]:
```
from getpass import getpass
print('登入中...')

username = input('Username: ')
password = getpass('Password: ')
print(f"使用者名稱: {username} 已登入")
```

```
登入中...
使用者名稱: mkao 已登入
```

9. 使用 `python -m` 將模組作為腳本執行

在命令提示字元中帶參數啟動進程

python -m 模組名稱

In [1]:
```
# 在 Jupyter notebook 單元格中使用 "!" 執行命令列指令
!python -m pip list
```

```
Package                   Version
------------------------- --------------
absl-py                   2.1.0
annotated-doc             0.0.3
annotated-types           0.7.0
```

```
anyio                       4.15.1
appdirs                     1.4.4
appnope                     0.1.4
argon2-cffi                 25.1.0
argon2-cffi-bindings        21.2.0
arrow                       1.3.0
asttokens                   3.0.1
async-lru                   2.0.5
attrs                       25.3.0
babel                       2.17.0
beautifulsoup4              4.13.4
bleach                      6.4.0
bs4                         0.0.2
certifi                     2026.4.22
cffi                        1.17.1
charset-normalizer          3.4.0
click                       8.1.8
comm                        0.2.3
contourpy                   1.3.1
cycler                      0.12.1
debugpy                     1.8.16
decorator                   5.2.1
defusedxml                  0.7.1
dnspython                   2.7.0
email_validator             2.2.0
et_xmlfile                  2.0.0
executing                   2.2.1
fastapi                     0.137.2
fastjsonschema              2.21.1
fibheap                     0.2.1
fonttools                   4.62.1
fqdn                        1.5.1
glog                        0.3.1
graphviz                    0.20.3
greenlet                    3.2.3
h11                         0.16.0
httpcore                    1.0.9
httpx                       0.28.1
idna                        3.18
ipykernel                   7.2.0
ipython                     9.11.0
ipython_pygments_lexers     1.1.1
isoduration                 20.11.0
jedi                        0.20.0
Jinja2                      3.1.6
json5                       0.12.0
jsonpickle                  4.1.2
jsonpointer                 3.0.0
jsonschema                  4.24.0
jsonschema-specifications   2025.4.1
jupyter_builder             1.2.2
jupyter_client              8.8.0
jupyter_core                5.9.1
jupyter-events              0.12.0
jupyter-lsp                 2.2.5
jupyter_server              2.20.0
```

```
jupyter_server_terminals  0.5.3
jupyterlab                4.6.3
jupyterlab_pygments       0.3.0
jupyterlab_server         2.28.0
kiwisolver                1.4.8
MarkupSafe                3.0.2
matplotlib                3.10.1
matplotlib-inline         0.2.1
mistune                   3.3.4
MouseInfo                 0.1.3
nbclient                  0.10.2
nbconvert                 7.17.1
nbformat                  5.10.4
nest_asyncio              1.6.0
networkx                  3.6.1
notebook                  7.5.6
notebook_shim             0.2.4
numpy                     2.1.3
openpyxl                  3.1.5
overrides                 7.7.0
packaging                 24.2
pandas                    2.2.3
pandoc                    2.4
pandocfilters             1.5.1
parso                     0.8.7
pexpect                   4.9.0
pillow                    12.3.0
pip                       26.2.1
platformdirs              4.9.4
playwright                1.52.0
plumbum                   1.9.0
ply                       3.11
prometheus_client         0.22.1
prompt_toolkit            3.0.52
psutil                    7.0.0
ptyprocess                0.7.0
pure_eval                 0.2.3
PyAutoGUI                 0.9.54
pycparser                 2.22
pydantic                  2.11.3
pydantic_core             2.33.1
pyee                      13.0.1
pygame                    2.6.1
PyGetWindow               0.0.9
Pygments                  2.20.0
PyMsgBox                  1.0.9
pyobjc-core               11.0
pyobjc-framework-Cocoa    11.0
pyobjc-framework-Quartz   11.0
pyparsing                 3.2.3
pyperclip                 1.9.0
PyQt5                     5.15.11
PyQt5-Qt5                 5.15.17
PyQt5_sip                 12.17.0
PyRect                    0.2.0
PyScreeze                 1.0.1
```

```
python-dateutil           2.9.0.post0
python-dotenv             1.2.2
python-gflags             3.1.2
python-json-logger        3.3.0
pytweening                1.2.0
pytz                      2024.2
pyvis                     0.3.2
PyYAML                    6.0.2
pyzmq                     27.1.0
qrcode                    8.1
referencing               0.36.2
requests                  2.33.1
rfc3339-validator         0.1.4
rfc3986-validator         0.1.1
rpds-py                   0.25.1
rubicon-objc              0.5.0
schedule                  1.2.2
scipy                     1.17.0
Send2Trash                1.8.3
setuptools                84.0.0
six                       1.16.0
sniffio                   1.3.1
sortedcollections         2.1.0
sortedcontainers          2.4.0
soupsieve                 2.9.2
stack_data                0.6.3
starlette                 1.3.1
terminado                 0.18.1
tinycss2                  1.4.0
tornado                   6.5.8
tqdm                      4.67.1
traitlets                 5.15.0
types-python-dateutil     2.9.0.20250516
typing_extensions         4.16.0
typing-inspection         0.4.2
tzdata                    2024.2
uri-template              1.3.0
urllib3                   2.7.0
uv                        0.11.23
uvicorn                   0.34.1
wcwidth                   0.2.14
webcolors                 24.11.1
webencodings              0.5.1
websocket-client          1.8.0
websockets                10.4
wheel                     0.46.3
xlrd                      2.0.1
```

10. 使用 `help`/`dir` 查看文檔字串和物件屬性

In [50]:
```python
import uvicorn

# 查看物件屬性
help(uvicorn)
```

```
Help on package uvicorn:

NAME
    uvicorn

PACKAGE CONTENTS
    __main__
    _subprocess
    _types
    config
    importer
    lifespan (package)
    logging
    loops (package)
    main
    middleware (package)
    protocols (package)
    server
    supervisors (package)
    workers

CLASSES
    builtins.object
        uvicorn.config.Config
        uvicorn.server.Server

    class Config(builtins.object)
     |  Config(
     |      app: 'ASGIApplication | Callable[..., Any] | str',
     |      host: 'str' = '127.0.0.1',
     |      port: 'int' = 8000,
     |      uds: 'str | None' = None,
     |      fd: 'int | None' = None,
     |      loop: 'LoopSetupType' = 'auto',
     |      http: 'type[asyncio.Protocol] | HTTPProtocolType' = 'aut
o',
     |      ws: 'type[asyncio.Protocol] | WSProtocolType' = 'auto',
     |      ws_max_size: 'int' = 16777216,
     |      ws_max_queue: 'int' = 32,
     |      ws_ping_interval: 'float | None' = 20.0,
     |      ws_ping_timeout: 'float | None' = 20.0,
     |      ws_per_message_deflate: 'bool' = True,
     |      lifespan: 'LifespanType' = 'auto',
     |      env_file: 'str | os.PathLike[str] | None' = None,
     |      log_config: 'dict[str, Any] | str | RawConfigParser | I
O[Any] | None' = {'version': 1, 'disable_existing_loggers': False, '
formatters': {'default': {'()': 'uvicorn.logging.DefaultFormatter',
'fmt': '%(levelprefix)s %(message)s', 'use_colors': None}, 'access':
{'()': 'uvicorn.logging.AccessFormatter', 'fmt': '%(levelprefix)s %(
client_addr)s - "%(request_line)s" %(status_code)s'}}, 'handlers':
{'default': {'formatter': 'default', 'class': 'logging.StreamHandle
r', 'stream': 'ext://sys.stderr'}, 'access': {'formatter': 'access',
'class': 'logging.StreamHandler', 'stream': 'ext://sys.stdout'}}, 'l
oggers': {'uvicorn': {'handlers': ['default'], 'level': 'INFO', 'pro
pagate': False}, 'uvicorn.error': {'level': 'INFO'}, 'uvicorn.acces
s': {'handlers': ['access'], 'level': 'INFO', 'propagate': False}}},
```

```
 |      log_level: 'str | int | None' = None,
 |      access_log: 'bool' = True,
 |      use_colors: 'bool | None' = None,
 |      interface: 'InterfaceType' = 'auto',
 |      reload: 'bool' = False,
 |      reload_dirs: 'list[str] | str | None' = None,
 |      reload_delay: 'float' = 0.25,
 |      reload_includes: 'list[str] | str | None' = None,
 |      reload_excludes: 'list[str] | str | None' = None,
 |      workers: 'int | None' = None,
 |      proxy_headers: 'bool' = True,
 |      server_header: 'bool' = True,
 |      date_header: 'bool' = True,
 |      forwarded_allow_ips: 'list[str] | str | None' = None,
 |      root_path: 'str' = '',
 |      limit_concurrency: 'int | None' = None,
 |      limit_max_requests: 'int | None' = None,
 |      backlog: 'int' = 2048,
 |      timeout_keep_alive: 'int' = 5,
 |      timeout_notify: 'int' = 30,
 |      timeout_graceful_shutdown: 'int | None' = None,
 |      callback_notify: 'Callable[..., Awaitable[None]] | None'
= None,
 |      ssl_keyfile: 'str | os.PathLike[str] | None' = None,
 |      ssl_certfile: 'str | os.PathLike[str] | None' = None,
 |      ssl_keyfile_password: 'str | None' = None,
 |      ssl_version: 'int' = <_SSLMethod.PROTOCOL_TLS_SERVER: 17
>,
 |      ssl_cert_reqs: 'int' = <VerifyMode.CERT_NONE: 0>,
 |      ssl_ca_certs: 'str | None' = None,
 |      ssl_ciphers: 'str' = 'TLSv1',
 |      headers: 'list[tuple[str, str]] | None' = None,
 |      factory: 'bool' = False,
 |      h11_max_incomplete_event_size: 'int | None' = None
 |  )
 |
 |  Methods defined here:
 |
 |  __init__(
 |      self,
 |      app: 'ASGIApplication | Callable[..., Any] | str',
 |      host: 'str' = '127.0.0.1',
 |      port: 'int' = 8000,
 |      uds: 'str | None' = None,
 |      fd: 'int | None' = None,
 |      loop: 'LoopSetupType' = 'auto',
 |      http: 'type[asyncio.Protocol] | HTTPProtocolType' = 'aut
o',
 |      ws: 'type[asyncio.Protocol] | WSProtocolType' = 'auto',
 |      ws_max_size: 'int' = 16777216,
 |      ws_max_queue: 'int' = 32,
 |      ws_ping_interval: 'float | None' = 20.0,
 |      ws_ping_timeout: 'float | None' = 20.0,
 |      ws_per_message_deflate: 'bool' = True,
 |      lifespan: 'LifespanType' = 'auto',
 |      env_file: 'str | os.PathLike[str] | None' = None,
```

```
     |      log_config: 'dict[str, Any] | str | RawConfigParser | I
O[Any] | None' = {'version': 1, 'disable_existing_loggers': False, '
formatters': {'default': {'()': 'uvicorn.logging.DefaultFormatter',
'fmt': '%(levelprefix)s %(message)s', 'use_colors': None}, 'access':
{'()': 'uvicorn.logging.AccessFormatter', 'fmt': '%(levelprefix)s %(
client_addr)s - "%(request_line)s" %(status_code)s'}}, 'handlers':
{'default': {'formatter': 'default', 'class': 'logging.StreamHandle
r', 'stream': 'ext://sys.stderr'}, 'access': {'formatter': 'access',
'class': 'logging.StreamHandler', 'stream': 'ext://sys.stdout'}}, 'l
oggers': {'uvicorn': {'handlers': ['default'], 'level': 'INFO', 'pro
pagate': False}, 'uvicorn.error': {'level': 'INFO'}, 'uvicorn.acces
s': {'handlers': ['access'], 'level': 'INFO', 'propagate': False}}},
     |      log_level: 'str | int | None' = None,
     |      access_log: 'bool' = True,
     |      use_colors: 'bool | None' = None,
     |      interface: 'InterfaceType' = 'auto',
     |      reload: 'bool' = False,
     |      reload_dirs: 'list[str] | str | None' = None,
     |      reload_delay: 'float' = 0.25,
     |      reload_includes: 'list[str] | str | None' = None,
     |      reload_excludes: 'list[str] | str | None' = None,
     |      workers: 'int | None' = None,
     |      proxy_headers: 'bool' = True,
     |      server_header: 'bool' = True,
     |      date_header: 'bool' = True,
     |      forwarded_allow_ips: 'list[str] | str | None' = None,
     |      root_path: 'str' = '',
     |      limit_concurrency: 'int | None' = None,
     |      limit_max_requests: 'int | None' = None,
     |      backlog: 'int' = 2048,
     |      timeout_keep_alive: 'int' = 5,
     |      timeout_notify: 'int' = 30,
     |      timeout_graceful_shutdown: 'int | None' = None,
     |      callback_notify: 'Callable[..., Awaitable[None]] | None'
= None,
     |      ssl_keyfile: 'str | os.PathLike[str] | None' = None,
     |      ssl_certfile: 'str | os.PathLike[str] | None' = None,
     |      ssl_keyfile_password: 'str | None' = None,
     |      ssl_version: 'int' = <_SSLMethod.PROTOCOL_TLS_SERVER: 17
>,
     |      ssl_cert_reqs: 'int' = <VerifyMode.CERT_NONE: 0>,
     |      ssl_ca_certs: 'str | None' = None,
     |      ssl_ciphers: 'str' = 'TLSv1',
     |      headers: 'list[tuple[str, str]] | None' = None,
     |      factory: 'bool' = False,
     |      h11_max_incomplete_event_size: 'int | None' = None
     |  )
     |      Initialize self.  See help(type(self)) for accurate sign
ature.
     |
     |  bind_socket(self) -> 'socket.socket'
     |
     |  configure_logging(self) -> 'None'
     |
     |  load(self) -> 'None'
     |
```

```
      |  setup_event_loop(self) -> 'None'
      |
      |  ----------------------------------------------------------------
----------
      |  Readonly properties defined here:
      |
      |  asgi_version
      |
      |  is_ssl
      |
      |  should_reload
      |
      |  use_subprocess
      |
      |  ----------------------------------------------------------------
----------
      |  Data descriptors defined here:
      |
      |  __dict__
      |      dictionary for instance variables
      |
      |  __weakref__
      |      list of weak references to the object

    class Server(builtins.object)
      |  Server(config: 'Config') -> 'None'
      |
      |  Methods defined here:
      |
      |  __init__(self, config: 'Config') -> 'None'
      |      Initialize self.  See help(type(self)) for accurate sign
ature.
      |
      |  capture_signals(self) -> 'Generator[None, None, None]'
      |
      |  handle_exit(self, sig: 'int', frame: 'FrameType | None') ->
'None'
      |
      |  async main_loop(self) -> 'None'
      |
      |  async on_tick(self, counter: 'int') -> 'bool'
      |
      |  run(self, sockets: 'list[socket.socket] | None' = None) -> '
None'
      |
      |  async serve(self, sockets: 'list[socket.socket] | None' = No
ne) -> 'None'
      |
      |  async shutdown(self, sockets: 'list[socket.socket] | None' =
None) -> 'None'
      |
      |  async startup(self, sockets: 'list[socket.socket] | None' =
None) -> 'None'
      |
      |  ----------------------------------------------------------------
----------
```

```
 |  Data descriptors defined here:
 |
 |  __dict__
 |      dictionary for instance variables
 |
 |  __weakref__
 |      list of weak references to the object

FUNCTIONS
    run(
        app: 'ASGIApplication | Callable[..., Any] | str',
        *,
        host: 'str' = '127.0.0.1',
        port: 'int' = 8000,
        uds: 'str | None' = None,
        fd: 'int | None' = None,
        loop: 'LoopSetupType' = 'auto',
        http: 'type[asyncio.Protocol] | HTTPProtocolType' = 'auto',
        ws: 'type[asyncio.Protocol] | WSProtocolType' = 'auto',
        ws_max_size: 'int' = 16777216,
        ws_max_queue: 'int' = 32,
        ws_ping_interval: 'float | None' = 20.0,
        ws_ping_timeout: 'float | None' = 20.0,
        ws_per_message_deflate: 'bool' = True,
        lifespan: 'LifespanType' = 'auto',
        interface: 'InterfaceType' = 'auto',
        reload: 'bool' = False,
        reload_dirs: 'list[str] | str | None' = None,
        reload_includes: 'list[str] | str | None' = None,
        reload_excludes: 'list[str] | str | None' = None,
        reload_delay: 'float' = 0.25,
        workers: 'int | None' = None,
        env_file: 'str | os.PathLike[str] | None' = None,
        log_config: 'dict[str, Any] | str | RawConfigParser | IO[An
y] | None' = {'version': 1, 'disable_existing_loggers': False, 'form
atters': {'default': {'()': 'uvicorn.logging.DefaultFormatter', 'fm
t': '%(levelprefix)s %(message)s', 'use_colors': None}, 'access':
{'()': 'uvicorn.logging.AccessFormatter', 'fmt': '%(levelprefix)s %(
client_addr)s - "%(request_line)s" %(status_code)s'}}, 'handlers':
{'default': {'formatter': 'default', 'class': 'logging.StreamHandle
r', 'stream': 'ext://sys.stderr'}, 'access': {'formatter': 'access',
'class': 'logging.StreamHandler', 'stream': 'ext://sys.stdout'}}, 'l
oggers': {'uvicorn': {'handlers': ['default'], 'level': 'INFO', 'pro
pagate': False}, 'uvicorn.error': {'level': 'INFO'}, 'uvicorn.acces
s': {'handlers': ['access'], 'level': 'INFO', 'propagate': False}}},
        log_level: 'str | int | None' = None,
        access_log: 'bool' = True,
        proxy_headers: 'bool' = True,
        server_header: 'bool' = True,
        date_header: 'bool' = True,
        forwarded_allow_ips: 'list[str] | str | None' = None,
        root_path: 'str' = '',
        limit_concurrency: 'int | None' = None,
        backlog: 'int' = 2048,
        limit_max_requests: 'int | None' = None,
        timeout_keep_alive: 'int' = 5,
```

```
            timeout_graceful_shutdown: 'int | None' = None,
            ssl_keyfile: 'str | os.PathLike[str] | None' = None,
            ssl_certfile: 'str | os.PathLike[str] | None' = None,
            ssl_keyfile_password: 'str | None' = None,
            ssl_version: 'int' = <_SSLMethod.PROTOCOL_TLS_SERVER: 17>,
            ssl_cert_reqs: 'int' = <VerifyMode.CERT_NONE: 0>,
            ssl_ca_certs: 'str | None' = None,
            ssl_ciphers: 'str' = 'TLSv1',
            headers: 'list[tuple[str, str]] | None' = None,
            use_colors: 'bool | None' = None,
            app_dir: 'str | None' = None,
            factory: 'bool' = False,
            h11_max_incomplete_event_size: 'int | None' = None
        ) -> 'None'

    DATA
        __all__ = ['main', 'run', 'Config', 'Server']
        main = <Command main>

    VERSION
        0.34.1

    FILE
        /Users/michaelkao/miniconda3/envs/pex/lib/python3.13/site-packag
    es/uvicorn/__init__.py
```

In [51]:
```
import uvicorn
# 查看物件屬性
dir(uvicorn)
```

Out[51]:
```
['Config',
 'Server',
 '__all__',
 '__builtins__',
 '__cached__',
 '__doc__',
 '__file__',
 '__loader__',
 '__name__',
 '__package__',
 '__path__',
 '__spec__',
 '__version__',
 '_subprocess',
 '_types',
 'config',
 'importer',
 'logging',
 'main',
 'middleware',
 'run',
 'server',
 'supervisors']
```

In [52]:
```
import uvicorn

# 在物件後加上 "?" 可以直接查看相關文件檔案位置
uvicorn?
```

```
Type:        module
String form: <module 'uvicorn' from '/Users/michaelkao/miniconda3/envs/pex/lib/python3.13/site-packages/uvicorn/__init__.py'>
File:        ~/miniconda3/envs/pex/lib/python3.13/site-packages/uvicorn/__init__.py
Docstring:   <no docstring>
```

In [53]:
```
from datetime import datetime

# 尋找函數物件的屬性和方法
dir(datetime)
```

Out[53]:
```
['__add__',
 '__class__',
 '__delattr__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattribute__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__init__',
 '__init_subclass__',
 '__le__',
 '__lt__',
 '__ne__',
 '__new__',
 '__radd__',
 '__reduce__',
 '__reduce_ex__',
 '__replace__',
 '__repr__',
 '__rsub__',
 '__setattr__',
 '__sizeof__',
 '__str__',
 '__sub__',
 '__subclasshook__',
 'astimezone',
 'combine',
 'ctime',
 'date',
 'day',
 'dst',
 'fold',
 'fromisocalendar',
 'fromisoformat',
 'fromordinal',
```

```
 'fromtimestamp',
 'hour',
 'isocalendar',
 'isoformat',
 'isoweekday',
 'max',
 'microsecond',
 'min',
 'minute',
 'month',
 'now',
 'replace',
 'resolution',
 'second',
 'strftime',
 'strptime',
 'time',
 'timestamp',
 'timetuple',
 'timetz',
 'today',
 'toordinal',
 'tzinfo',
 'tzname',
 'utcfromtimestamp',
 'utcnow',
 'utcoffset',
 'utctimetuple',
 'weekday',
 'year']
```

In [54]:
```
from datetime import datetime

# 這是什麼？
print(datetime.today)

# 執行它
print(datetime.today())
```

```
<built-in method today of type object at 0x101ff48b0>
2026-02-03 11:53:11.799638
```

11. 使用 `Iterables` 進行迭代

可迭代物件(Iterable)是一個可以進行迭代的物件。當傳遞給 iter() 方法時，它會生成一個迭代器(Iterator)。

迭代器是一個物件，用於通過 **next**() 方法對可迭代物件進行迭代。

迭代器具有 **next**() 方法，該方法返回物件的下一個項目。請注意，每個迭代器都是一個可迭代物件，但並非每個可迭代物件都是迭代器。

例如，列表(list)是可迭代的，但列表本身不是迭代器。可以通過 iter() 函數從可迭

代物件創建迭代器。為了實現這一點，物件的類需要一個返回迭代器的 **iter** 方法，或者一個從 0 開始的連續索引的 **getitem** 方法。

In [55]:
```
for element in ['c123', 142_857, 'Michael', 'Kao']:
    print(element, end=" ")
print()

for element in ('c123', 142_857, 'Michael', 'Kao'):
    print(element, end=" ")
print()

for letter in 'Michael Kao':
    print(letter, end=" ")
print()

for byte in b'Binary':
    print(byte, end=" ")
print()

# 上面展示的所有物件都是可迭代的
# 然而，整數型別是不可迭代的，如下所示
c = 142_857

# 必須轉換為字串才能讓整數可迭代
for digit in str(c):
    print(digit, end=" ")
print()

# 或者轉換為列表讓整數可迭代
digits = [int(d) for d in str(c)]
for digit in digits:
    print(digit, end=" ")
print()

# 導入必要的模組
import itertools

# 建立牌面數字清單：2-10 加上 J, Q, K, A
ranks = list(range(2, 11)) + ["J", "Q", "K", "A"]
# 將所有元素轉換為字串
ranks = [str(rank) for rank in ranks]

print(f'牌面數字: {ranks}')

# 定義四種花色
suits = ['♠', '♥', '♦', '♣']

# 使用 itertools.product 建立所有可能的牌面組合（花色 + 數字）
deck = [card for card in itertools.product(suits, ranks)]

# 遍歷並顯示前52張牌
for (index, card) in enumerate(deck):
    # 顯示牌組中的每一張牌，編號從1開始
    print(f'{1+index}: {card}')
    # 安全機制：最多只顯示52張牌
```

```
    if index >= 51:  # 因為索引從0開始，所以51表示第52張牌
        break

# 計算從52張牌中選取5張的所有可能組合
# 注意：這個計算量非常大（2,598,960種組合）
hands = [hand for hand in itertools.combinations(deck, 5)]
print(f'五張牌的撲克牌組合共有 {len(hands):,} 種。')
```

```
c123 142857 Michael Kao
c123 142857 Michael Kao
M i c h a e l   K a o
66 105 110 97 114 121
1 4 2 8 5 7
1 4 2 8 5 7
牌面數字: ['2', '3', '4', '5', '6', '7', '8', '9', '10', 'J', 'Q', '
K', 'A']
1: ('♠', '2')
2: ('♠', '3')
3: ('♠', '4')
4: ('♠', '5')
5: ('♠', '6')
6: ('♠', '7')
7: ('♠', '8')
8: ('♠', '9')
9: ('♠', '10')
10: ('♠', 'J')
11: ('♠', 'Q')
12: ('♠', 'K')
13: ('♠', 'A')
14: ('♥', '2')
15: ('♥', '3')
16: ('♥', '4')
17: ('♥', '5')
18: ('♥', '6')
19: ('♥', '7')
20: ('♥', '8')
21: ('♥', '9')
22: ('♥', '10')
23: ('♥', 'J')
24: ('♥', 'Q')
25: ('♥', 'K')
26: ('♥', 'A')
27: ('♦', '2')
28: ('♦', '3')
29: ('♦', '4')
30: ('♦', '5')
31: ('♦', '6')
32: ('♦', '7')
33: ('♦', '8')
34: ('♦', '9')
35: ('♦', '10')
36: ('♦', 'J')
37: ('♦', 'Q')
38: ('♦', 'K')
39: ('♦', 'A')
40: ('♣', '2')
```

```
41: ('♣', '3')
42: ('♣', '4')
43: ('♣', '5')
44: ('♣', '6')
45: ('♣', '7')
46: ('♣', '8')
47: ('♣', '9')
48: ('♣', '10')
49: ('♣', 'J')
50: ('♣', 'Q')
51: ('♣', 'K')
52: ('♣', 'A')
五張牌的撲克牌組合共有 2,598,960 種。
```